

Attention and transformers

Dataset: IMDb film reviews

How to use these notes

The primary text for this lecture is Géron, Chapters 14–15. These notes are not a replacement for it and do not repeat it: they are the lecture’s own argument written out at length — the derivation done slowly, the numbers with the runs that produced them, and the reasoning between one slide and the next that a deck can only gesture at. Read the chapter for the method; read these for why the lecture makes the choices it does. Every figure quoted here is printed by this lecture’s notebook, and the course checks that mechanically rather than trusting it.

Contents

1	One change removes three problems	2
2	Derivation: scaled dot-product attention	2
2.1	Heads, and positions	3
2.2	One block, and what masking is for	3
3	Fine-tuning a pretrained transformer	4
3.1	Where the pretrained model earns its place	5
4	The same representation, two more deliverables	5
5	One more prompt, and this is the one to remember	6
5.1	The text leak that really is expensive	7
6	Is 90.6% good?	8
7	The notebook	8
8	Exercises	8

1 One change removes three problems

Lecture 16 ended with three things recurrence cannot do, and Lecture 17 built a classifier that ran into all of them: the computation is sequential along the sequence; everything remembered must fit through one fixed-width state; and position 400 is four hundred steps from position 2, however good the gates.

Let every position compute a weighted sum over every position *directly*, with the weights learned from the content. No recurrence, no single state, and every pair of positions one step apart.

2 Derivation: scaled dot-product attention

The output at position i is $\sum_j w_{ij} \mathbf{v}_j$ — a weighted sum of values, one per position. Note what is already gone: there is no recurrence in that expression, so nothing forces the positions to be computed in order. Everything below is about where w_{ij} comes from.

Project each position three ways with learned matrices. A **query** is what this position is looking for; a **key** is what that position offers; a **value** is what it contributes if chosen. Score every pair by $s_{ij} = \mathbf{q}_i^\top \mathbf{k}_j$ — a dot product, because it is the cheapest similarity a matrix multiply computes for all pairs at once — then normalise along the sequence with Lecture 17's softmax and take the weighted sum. For all positions at once: $\text{softmax}(\mathbf{QK}^\top) \mathbf{V}$.

Why divide by $\sqrt{d_k}$

Take $\mathbf{q}$ and $\mathbf{k}$ to have independent components with mean 0 and variance 1. Then $s = \sum_{d=1}^{d_k} q_d k_d$ is a sum of d_k independent zero-mean terms, each of variance $\text{Var}(q_d) \text{Var}(k_d) = 1$, so

$$\text{Var}(s) = d_k.$$

The scores grow with the dimension: at $d_k = 64$, typical scores are already of order 8. Softmax of large scores saturates — one weight goes to 1, the rest to 0, and its gradient goes to zero with them, which is Lecture 11's vanishing gradient arriving through a different door.

Dividing by $\sqrt{d_k}$ returns the variance to 1 whatever the dimension:

$$\text{Attention} = \text{softmax}\left(\frac{\mathbf{QK}^\top}{\sqrt{d_k}}\right) \mathbf{V}.$$

The scale is not a hyperparameter. It is the number that makes the score distribution independent of d_k .

The trade, stated plainly

Every pair of positions interacts directly, so the path length between any two is **1**, against $O(T)$ for a recurrent net — and all positions compute at once, so the sequence axis parallelises. The cost is $O(T^2)$ in time and memory.

Recurrence is linear in T and sequential, with a long gradient path. Attention is quadratic in T and parallel, with a path of length one. On the sequence lengths people actually use the second wins — and when it does not, the T^2 term is what has to be approximated.

2.1 Heads, and positions

One attention layer computes one notion of relevance. Run h of them on different learned projections, each of width d_{model}/h so that h heads cost the same as one wide one, and concatenate. Different heads learn different relations — adjacency, agreement, coreference — and nothing tells them to; the division of labour is learned. That is Lecture 7's ensemble argument in a new place: several estimators of the same thing beat one, provided their errors are not identical, and different projections are what stops them being identical.

The problem attention creates

$\text{softmax}(\mathbf{QK}^T)\mathbf{V}$ is *permutation equivariant*: shuffle the input positions and the outputs shuffle with them, unchanged.

So the model literally cannot tell “the film was not good” from “good, the film was not”. Recurrence got order for free by processing in order. Attention has to be *told*.

Positional encoding adds a position-dependent vector before the first layer: *sinusoidal*, fixed and extending to unseen lengths; *learned*, an embedding table indexed by position; or *relative*, encoding the distance between two positions in the score, which is what most current models do.

2.2 One block, and what masking is for

A transformer block is multi-head self-attention, then add the input back and normalise, then a position-wise feed-forward network applied identically at every position, then add and normalise again. Both wrappers are Lecture 11 unchanged: the residual gives the gradient an additive path through depth — the same argument as an LSTM gate, one level up — and the normalisation keeps the variance from drifting as blocks stack. *A transformer is deep, and it is deep only because of these two.*

A decoder generating text must not look at what it has not written, so $s_{ij} = -\infty$ for $j > i$ before the softmax. $e^{-\infty} = 0$, so those positions get exactly zero weight and contribute nothing — and, crucially, no gradient either. That is Lecture 15's causality condition again, enforced in the architecture rather than in the data split: *an unmasked decoder is a model that has read the answer.*

Family	Attention	Good at
Encoder-only	bidirectional	classification, tagging, retrieval
Decoder-only	masked, causal	generation
Encoder-decoder	both, plus cross-attention	translation, summarisation

The choice is the same question as bidirectional layers in Lecture 16: at prediction time, do I have the whole input? Today we classify a review we can see in full, so encoder-only is right.

Self-attention and cross-attention are the *same arithmetic* — only where **Q** and **K** come from changes, which is why one implementation serves both.

Where the parameters are

Attention is $4d_{\text{model}}^2$ per layer, for the four projections; the feed-forward network is $8d_{\text{model}}^2$, being a fourfold expansion and back.

Two thirds of the parameters in a transformer block are in the feed-forward, not in the attention. The attention is what makes it work and it is the minority of the weights — a useful corrective to the name.

Attention itself is six lines of code, and every one is a step of the derivation. The rest of a transformer is arrangement around that function. Which is worth saying plainly: *the architecture that changed the field is, at its centre, a similarity, a softmax and a weighted average.*

Implement it and check it against the library. The usual causes of disagreement are a transposed key matrix, a mask applied after the softmax instead of before, or a missing scale — and each of those *runs*. Only the comparison finds them.

3 Fine-tuning a pretrained transformer

The model we borrow has 66,955,010 parameters, the same 30,522-token vocabulary as last lecture, an embedding width of 768, and was pretrained to predict masked words. *Last lecture we took $30,522 \times 768$ numbers out of this model and threw the rest away. The rest is where the language is.*

Fine-tuning puts a new, randomly initialised head on a pretrained body, at a learning rate fifty times smaller than we used from scratch — large steps destroy what was pretrained — for one pass over the data rather than four.

How much is the head, and how much is the body?

The same backbone with its untrained head and no fine-tuning scores **40.1%**, against 50.0% for always predicting one class, on 5,000 test reviews of which 49.8% are positive.

Note it is *below* the majority class: an untrained head is an arbitrary hyperplane and which side it puts each class on is a coin toss. The pretraining on its own predicts nothing about sentiment, because nothing has told it what the two output units mean.

Model	Test accuracy
Always one class	50.0%
tf-idf + logistic regression	90.1%
GRU from scratch	83.4%
GRU, pretrained embeddings	84.2%
DistilBERT, fine-tuned	90.6%

7.18 points over the model built last lecture — and only 0.45 points over the bag of words, which

is the second result of the afternoon and the less comfortable one.

Say it in errors, not in points

The GRU misclassifies 4,146 reviews out of 25,000; the fine-tuned transformer 2,352. *43% of the mistakes removed.*

Percentage points flatter a model near the top of the range. The error count is what the desk cares about, because each one is a review someone has to fix by hand.

3.1 Where the pretrained model earns its place

Labelled reviews	Bag of words	DistilBERT	Gap
200	75.7%	83.9%	8.12
1,000	83.0%	87.0%	3.98
5,000	88.2%	88.9%	0.74
20,000	90.5%	90.1%	-0.34

The gap is largest where labels are scarcest, which is the situation every real feedback desk starts in. *The pretraining is worth thousands of labels*, and that is the number to quote to whoever is paying for the annotation.

Each model's budget is chosen on the validation split at every size — the number of optimisation steps for the transformer, the regularisation strength for the linear model. Fixing the epochs instead would give the 200-review model thirteen steps and fixing the steps would undertrain the 20,000-review one, and measured, those two protocols disagree about the *sign* at 20,000 labels.

Model	Accuracy	Scoring 25,000	Per review
tf-idf + logistic regression	90.1%	3.6 s	0.14 ms
GRU from scratch	83.4%	6.3 s	0.25 ms
DistilBERT, fine-tuned	90.6%	133 s	5.30 ms

0.45 points for 37 times the cost per review — *a decision, not a result*. At 20,000 labelled reviews the desk would probably make it against us; at 200 it would not, and the table above says by how much. (The scoring time is the least reproducible number here: five passes ran from 103 s to 160 s. Quote it to two figures or not at all.)

4 The same representation, two more deliverables

The desk wanted two more things — “find me the ones like this one” and “tell me what people keep complaining about”. Neither is classification, and both fall out of the same pretrained models, because what those models produce is a *representation*, and a label was only ever one thing to do with it.

Mean-pooling over the *real* tokens gives one vector per review — and that pooling is last lecture's padding bug in its other costume: divide by the padded length instead of the true one and every short review is scaled towards zero. Normalising onto the unit sphere makes the dot product the cosine, so length carries no meaning and a long review cannot dominate every ranking.

Searching 5,000 negative reviews, embedded once in 11 seconds at 384 dimensions, is then one matrix product — no index, no training, no labels.

Query	Shared in the top 3
“the acting was wooden and unconvincing”	0 of 3
“the sound mix made the dialogue impossible to follow”	0 of 3
“far too long, it should have ended an hour earlier”	0 of 3

Not one document in common with keyword search, on any of the three. Keyword search can only retrieve documents that reuse the query’s words, and a complaint rarely uses the desk’s vocabulary.

That is a difference, not a victory

Nothing here measures which list is *better*. Judged by eye, on one of these three the keyword search returns the more relevant review. And cosine similarity has no notion of negation, so *the sound was perfect* and *the sound was not perfect* sit close together.

If you deploy this you owe yourself labelled queries and a retrieval metric — which is Lecture 19.

Clustering the same vectors is Lecture 8 unchanged, with k chosen by silhouette rather than by eye. Four groups, at a silhouette of 0.0179 — low in absolute terms, because complaint text does not fall into well-separated balls, and the groups are still legible: 1,936 reviews around *bad, worst, movies, acting*; 1,198 around *woman, character, women, role*; 1,170 around *horror, monster, zombie, scary*. Those terms are not the cluster centres — they are the words whose average tf-idf weight is highest inside the group relative to outside it, because *a group is named by what makes it different, not by what is common everywhere*.

5 One more prompt, and this is the one to remember

“Build a scikit-learn pipeline that vectorises the reviews with tf-idf and classifies them with logistic regression, and report the test accuracy.” Under-specified in the same place as Lecture 1’s — and this time the object being fitted is not a scaler.

The code that comes back calls `fit_transform` on *every* document and then splits. It runs, it splits, it never fits the classifier on test rows, and it reports a plausible number.

Reviewer question 2: what was fitted, and on what?

`TfidfVectorizer.fit` learns two things, and both come from the test rows here: *the vocabulary* — which terms get a column at all, so terms occurring only in test documents get their own feature — and *the inverse document frequencies*, computed over the whole corpus.

A scaler learns two numbers per column from an invertible affine map. A vectoriser learns *which columns exist*, and that is not a transformation of the features — it is a choice of feature space, made with the answers in the room.

Corpus	Leaky	Honest	Difference	sd over seeds
25,000 reviews	90.28%	90.22%	0.06 pts	0.07%
400 reviews	75.90%	75.60%	0.30 pts	2.53%

Read the last column before the one beside it. On the small corpus the mean gap is 0.30 points and the spread across seeds is eight times larger, and the leak wins on 10 of 20 seeds — exactly half. *This is a null result.*

Why so cheap? The leaky vocabulary really is bigger, 60,000 columns against 54,074. But a term occurring only in test documents has an all-zero column in the training rows, so logistic regression gives it coefficient exactly zero — there is no gradient to move it. What actually leaks is the inverse document frequencies, and an idf is an average over the whole corpus, so removing a quarter of the documents barely moves it. The leak changes the *scale* of features the model already had, and a regularised linear model is nearly indifferent to that.

And the decision rule is not about the size

Fit the vectoriser inside the pipeline anyway. Not because this leak is expensive — measured, it is not — but because *the next fitted object may not have that property*, and you will not re-derive the argument each time.

Three leaks, three small numbers: Lecture 2's scaler was worth nothing measurable, Lecture 10's per-batch mean 0.382 points, today's vectoriser nothing you can measure. A leaked score and an honest score can be identical, so you cannot tell from the number which one you are holding. The procedure is what separates them.

5.1 The text leak that really is expensive

The same document on both sides of the split. No pipeline protects you: every object is fitted correctly, on training rows only, and the test row is still one the model has memorised. On scraped or user-submitted text it is the normal case — a resubmit button is enough.

Checked on the official IMDb split: **123** test reviews are also present in training, and 96 reviews are duplicated within training. The corpus authors separated the *films* between train and test, and said so; they did not deduplicate the reviews. Three lines to run, and you now know rather than assume. *It is also exactly the check nobody ran, on the corpus everybody uses.*

123 out of 25,000 is too few to measure a cost with, so build a corpus where it is not: 1,500 reviews with three in ten submitted twice, 1,950 rows, and under a random split 35% of test rows have a copy of themselves in training.

Split	Reported accuracy	Spread over 10 seeds
random over rows	90.0%	1.87%
grouped by entry	82.17%	2.88%

7.83 points, and the leak wins on 10 of 10 seeds. Nothing was fitted on the test set in either case — *the difference is entirely which rows the split happened to separate*. Compare with the vectoriser leak: the object that was fitted wrongly cost far less than the rows that were split wrongly.

Failure	Diagnosed in	Measured cost
Scaler fitted before the split	Lecture 2	nothing measurable
Accuracy under class imbalance	Lecture 3	a useless model above 90%
Missing <code>zero_grad()</code>	Lecture 10	76.7 points
Random split on a time series	Lecture 15	an unreproducible score
Summarising at the padded last position	Lecture 17	18.24 points
Softmax before the loss	today	1.67 points
Vectoriser fitted before the split	today	0.30 points, not distinguishable from zero
Duplicate documents across the split	today	7.83 points

6 Is 90.6% good?

Against the desk’s own criterion: 2,352 reviews out of 25,000 are routed wrongly and nobody sees them again. Whether that is acceptable depends on what happens to a missed complaint, and we have not asked.

What a defensible report says

Quote the error count, not the percentage. Say which reviews are misread — short ones, sarcastic ones, ones whose verdict is in the last clause. Offer an abstain band around a probability of one half, and let the desk choose how wide it is.

The model is not the deliverable. The routing decision is.

What carried over unchanged: compute a trivial and a serious baseline before committing (Lecture 1); split first, and fitted objects see training rows only (Lecture 2); choose k by silhouette rather than by eye (Lecture 8); own the loop, and count a metric over the set (Lecture 10); and a pretrained representation beats a bigger architecture (Lecture 13).

7 The notebook

What to change

Shuffle the token positions of an input and run it through attention with and without positional encoding. Without, the output is the same multiset; with, it is not — permutation equivariance, demonstrated rather than asserted.

8 Exercises

1. Derive the variance of a dot-product attention score under the standard assumptions, and state precisely what dividing by $\sqrt{d_k}$ fixes. [5]
2. Explain why self-attention is permutation equivariant, and what follows for a model that must distinguish “not good” from “good . . . not”. [4]
3. A decoder is trained without a causal mask and reports excellent validation loss. State what

it has learned to do, and which earlier lecture's failure this is. [4]

4. A `TfidfVectorizer` is fitted before the split. Name the two quantities that leak, and explain why the measured cost is nevertheless near zero on a large corpus. [5]
5. A corpus contains duplicate documents and is split at random. Explain why no pipeline prevents the resulting leak, and give the split that does. [4]

Summary

Attention is three projections, a scaled dot product, a softmax and a weighted sum — and the scale is not a hyperparameter: the score variance is d_k , so dividing by $\sqrt{d_k}$ is what keeps the softmax off its saturated tails whatever the width. Every pair of positions is one step apart and the sequence axis parallelises, at a cost quadratic in length. Two thirds of a block's parameters are in the feed-forward rather than the attention.

Fine-tuning a 67-million-parameter pretrained body reached 90.6%: 7.18 points over the recurrent model, 43% of its errors removed — and only 0.45 points over tf-idf with logistic regression, at 37 times the cost per review. The pretraining is worth thousands of labels, and the gap closes as labels arrive: 8.12 points at 200 reviews, and -0.34 at 20,000.

Two leaks, measured against each other. A vectoriser fitted before the split learns the vocabulary and the idfs from the test rows, and costs 0.30 points on a small corpus with a seed spread eight times larger — a null result, and the rule stays procedural anyway, because the next fitted object may not be so forgiving. Duplicate documents across the split cost 7.83 points and win on 10 of 10 seeds, with nothing fitted wrongly at all. The official IMDb split contains 123 of them.